

HTTP 완벽 가이드 · Week 4

커넥션 재사용과 HTTP/2

4.5 ~ 4.8 · 10장 · p.104 ~ 121, 287 ~ 294

정우인

PART 1

HTTP/1.0+ 의 Keep-Alive 커넥션

4.5.2 ~ 4.5.6

왜 생겼나 — 요청마다 새 연결은 비싸다

1 handshake 비용

연결할 때마다 3-way handshake
→ 요청 전에 왕복 시간이 한 번 더

2 slow start

새 연결은 처음에 천천히 보낸다
→ 연결 초반엔 전송 속도가 느림

요청마다 새 연결



연결 = handshake + slow start · 매번 반복

Keep-Alive — 연결 재사용



한 번 맺은 TCP 연결을 여러 요청에서 재사용 하자 → HTTP/1.0+ 의 Keep-Alive

→ 연결 비용(handshake · slow start)을 한 번만 낸다

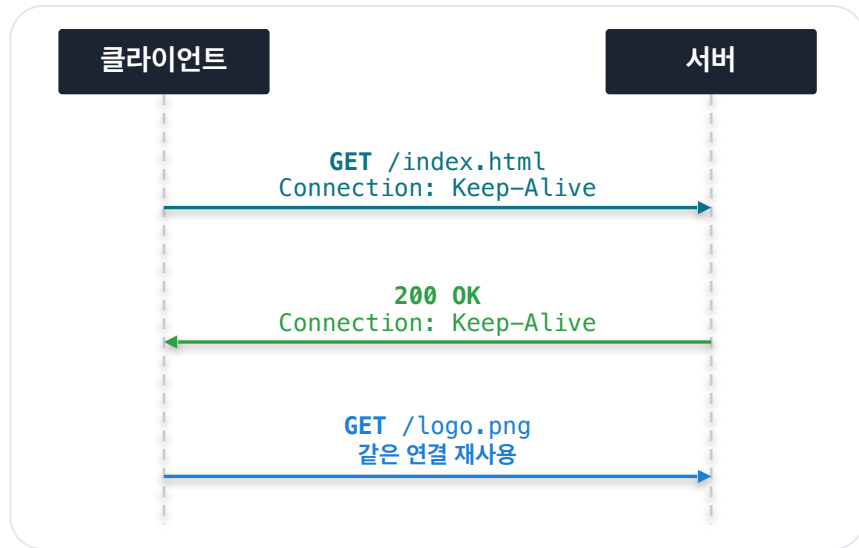
HTTP/1.0 — 기본은 꺼짐, 서로 합의해야 켜진다

1 클라이언트가 요청
Connection: Keep-Alive 헤더를 보냄

2 서버도 같은 헤더로 응답
→ 연결 유지

3 서버가 헤더 없이 응답
→ 응답이 끝나면 연결을 끊음

4 합의 후에도
클라이언트 · 서버 모두 언제든지 닫을 수 있음

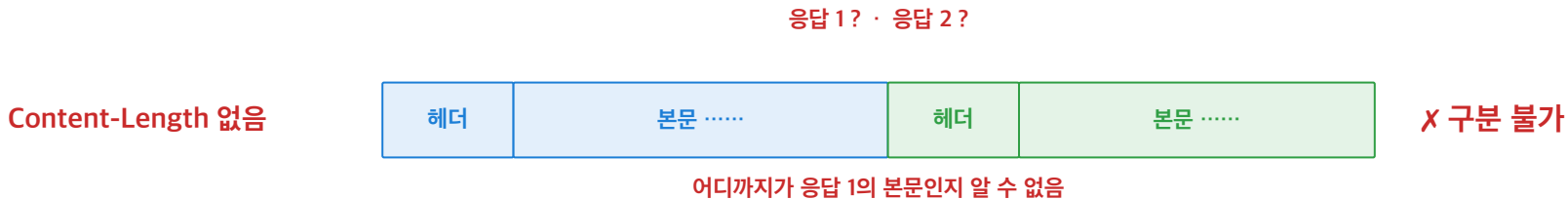
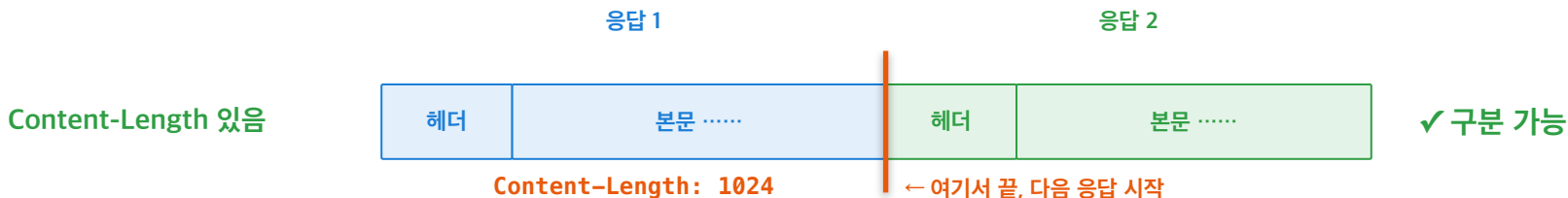


→ 양쪽이 같은 헤더를 주고받아야 연결이 유지된다

4.5.3 Keep-Alive 동작

연결을 유지하려면 본문 길이를 알아야 한다

하나의 TCP 연결 = 바이트가 이어지는 하나의 흐름 → 응답 1이 어디서 끝나는지 알아야 응답 2를 읽는다



→ 본문 길이를 모르면 TCP 버퍼에서 여러 요청 · 응답을 구분할 수 없다

Keep-Alive 헤더 — 연결을 언제까지 유지할까

Keep-Alive: **timeout=5**, **max=100**

timeout=5

연결을 얼마나 유지할지

5초 동안 요청이 없으면 → 연결 닫음

max=100

이 연결로 요청을 몇 개 더 받을지

요청 100개를 더 처리하면 → 연결 닫음

→ **시간(timeout)** 과 **요청 수(max)** 로 연결 유지 범위를 알려준다

멍청한 프록시 — Keep-Alive를 모르는 옛날 프록시

Connection: Keep-Alive 를 이해 못 하고 그대로 서버로 넘김

→ 브라우저 · 서버는 keep-alive라고 믿는데 **프록시만 모르는 상태**



프록시의 믿음 : “**연결 하나 = 요청 하나**”

1

연결 A에서
요청 하나를 읽는다

2

연결 B로
요청을 넘긴다

3

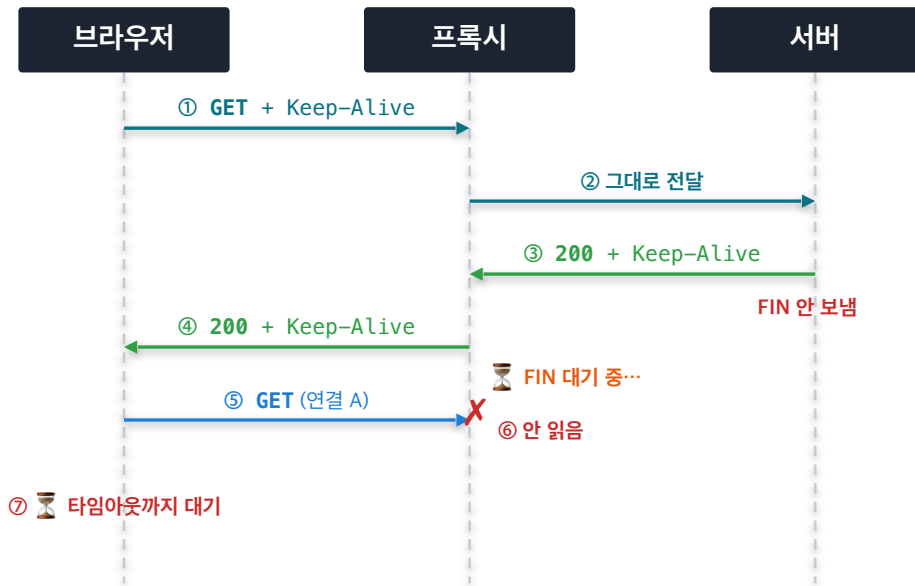
서버 응답을
브라우저로 넘긴다

4

서버가 FIN을 보내면
→ 응답 끝! A · B 닫기

→ 응답의 끝을 **서버의 FIN**으로만 판단한다 — FIN이 안 오면?

문제가 생기는 흐름 — 통신이 멈춘다



- 1 브라우저가 Connection: Keep-Alive를 붙여 요청
- 2 프록시는 헤더를 모른 채 그대로 서버에 전달
- 3 서버는 keep-alive 요청으로 착각 → FIN 없이 연결 유지
- 4 프록시는 응답을 넘기고 서버의 FIN을 계속 기다림
- 5 브라우저는 keep-alive 된 줄 알고 연결 A로 두 번째 요청
- 6 프록시는 한 연결에 요청이 또 올 줄 몰라 읽지 않음
- 7 브라우저는 응답을 못 받고 타임아웃까지 대기

→ 모두가 keep-alive라고 믿는데 **프록시만 몰라서** 두 번째 요청이 멈춘다

4-way handshake — 연결을 완전히 닫는 법

연결을 완전히 닫으려면
양쪽이 서로 확인을 주고받아야

FIN 나 보낼 거 다 보냈어

ACK 알겠어

한 방향마다 FIN + ACK
× 2 방향 = **신호 4번**



→ 각자 **FIN**을 보내고 **ACK**를 받아야 연결 종료가 끝난다

PART 2

HTTP/1.1 의 지속 커넥션

4.5.8 ~ 4.5.9

지속 커넥션 — Keep-Alive의 정식 표준

1

HTTP/1.0+ keep-alive를
정식 표준으로 만든 것

2

TCP 연결 하나를
여러 요청 · 응답에 재사용

3

가장 큰 차이
기본값이 뒤집혔다

HTTP/1.0+ Keep-Alive

기본값

OFF

켜려면

Connection: Keep-Alive

HTTP/1.1 지속 커넥션

기본값

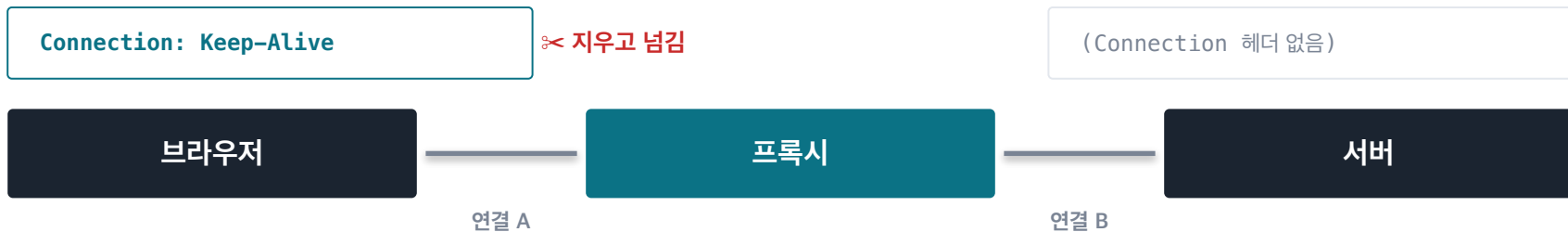
ON

끄려면

Connection: close

→ 기본값이 뒤집혔다 : **켜는 헤더** → **끄는 헤더**

멍청한 프록시에서 얻은 교훈 → 규칙으로



1 Connection 헤더는 바로 옆 구간에만

프록시는 이 헤더(와 여기에 나열된 헤더들)를 지우고 넘겨야 한다

2 연결 A · B를 각각 따로 관리

A를 유지할지, B를 유지할지
프록시가 구간마다 스스로 정한다

→ 프록시가 헤더를 그대로 넘기지 않으니 엇갈린 믿음이 생기지 않는다

보편화되었나? — 사실상 모든 곳의 기본값

지속 커넥션이 기본

✓ 브라우저

✓ 웹 서버

✓ 프록시

익숙한 예 — Tomcat (Spring Boot 내장 서버)

keepAliveTimeout

≈ timeout

요청 없이 연결을 얼마나 유지할지

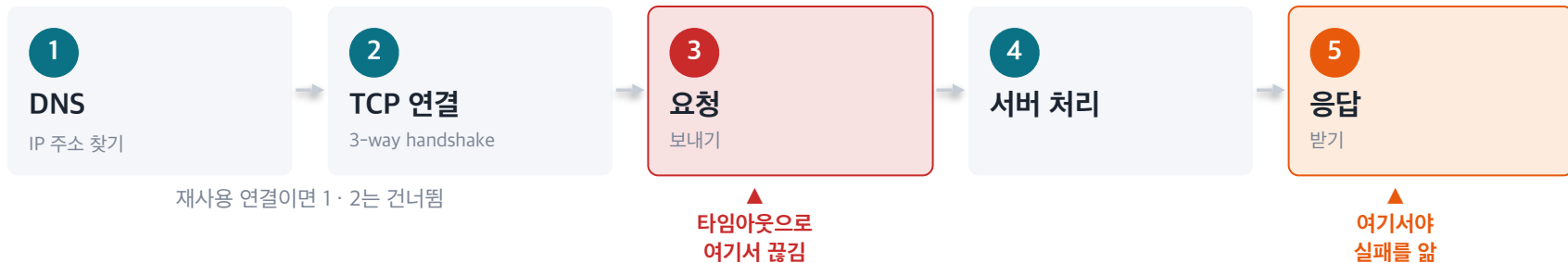
maxKeepAliveRequests

≈ max

한 연결로 요청을 몇 개까지 받을지 (기본 100)

→ 앞에서 본 **timeout** · **max** 가 서버 설정으로 그대로 살아 있다

프로그래머가 알아야 할 것 — 어디서 끊기나



문제는 클라이언트가 받는 모양

3단계까지 성공한 줄 알다가 5단계에서 실패를 안다

“요청 보냈는데
응답 읽다가 연결이 끊김”

→ 끊긴 건 요청 때인데, **에러는 응답을 읽을 때** 보인다

같은 에러, 두 가지 경우 — POST는 함부로 재시도 금지

경우 ①

지속 커넥션이 이미 끊겨 있었음

요청이 서버에 닿지 않음

결과 처리 안 됨

재시도해도 OK

경우 ②

서버가 처리 중 · 처리 후 죽거나 끊김

요청은 서버에 닿았음

결과 처리됐을 수 있음

재시도하면 2번 처리!

클라이언트 입장에서선 두 경우가 똑같은 에러 → 어느 쪽인지 구분할 수 없다

→ POST를 함부로 재시도하면 두 번 처리될 수 있다 (예: 주문 · 결제 2번)

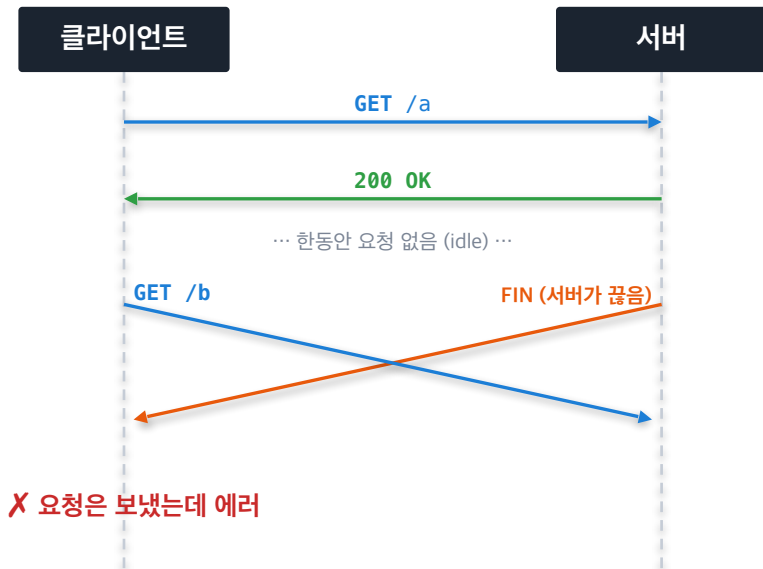
PART 3

커넥션 끊기에 대한 미스터리

4.7

4.7.1 마음대로 커넥션 끊기

누구든 언제든지 끊을 수 있다 — 하필 그 순간에



실무 예 — 외부 API 호출

우리 서버의 HTTP 클라이언트 커넥션 풀이
상대 서버가 이미 끊은 연결을 재사용 → 에러

해결 원칙

서버 keep-alive idle 타임아웃

클라이언트 idle 타임아웃

먼저 정리 →
끊긴 연결 안 씀

→ 클라이언트 idle 타임아웃을 **서버보다 짧게** 잡는다

본문의 끝을 알려줘야 한다 — 두 가지 방법

값이 틀리거나 없으면 → 받는 쪽은 데이터가 다 왔는지 판단할 수 없다

크기를 알 때

Content-Length: 1024

- 본문 크기를 정확히 알려준다
- 대부분의 일반 응답

or

크기를 모를 때

Transfer-Encoding: chunked

- 조각마다 크기를 붙여 나눠 보낸다
- AI 채팅 · 스트리밍 · 큰 파일 생성

둘 중 **하나만** 쓴다 · 요즘은 Spring 등 프레임워크가 알아서 넣어줘서 직접 다룰 일은 적다

→ 지속 커넥션은 연결이 열려 있으니 **본문의 끝**을 반드시 알려줘야 한다

chunked — 조각마다 크기를 붙여 보낸다

HTTP/1.1 200 OK ← 헤더는 한 번만
Transfer-Encoding: chunked

5 ← 크기 (16진수)

Hello

7

, World

0

← 끝

왜 필요한가

지속 커넥션에서 본문이 어디서 끝나는지 알아야 함

순서 · 누락

TCP가 보장 → HTTP는 신경 쓰지 않음

실무

Spring 등 프레임워크가 자동 처리

HTTP/2부터

원래 프레임 단위로 나눠 보내서 이 헤더를 안 씀

→ 크기를 모르면 **조각 + 크기** 로 보내고, 크기 **0** 으로 끝을 알린다

응답이 안 왔다면 — 다시 보내도 될까?

연결은 에러 없이도
언제든 끊길 수 있다



애플리케이션이
대비해야 한다



성공 · 실패를 모르면
재시도해도 되는 것만 재전송

GET

조회

반복해도 영향 없다

✓ 재시도 OK

POST

결제 · 송금 · 주문

반복하면 중복 주문이 생긴다

✗ 자동 재시도는 피한다

여러 번 해도 결과가 같은 성질 = 멱등성 (idempotent)

→ 멱등한 요청만 자동으로 재시도한다

실무에서는 — 요청 쪽과 제공 쪽이 함께 막는다

API 요청 쪽

결과를 세 가지로 나눈다

성공

실패

모름

→ 후처리

- 즉시 재요청 · 일정 시간 뒤 재시도
- 성공 여부 확인 후 재시도
- 결제 취소 (보상 트랜잭션) · 수기 처리

후처리도 실패하면 → “모름” 상태로 저장
나중에 다시 처리

API 제공 쪽

먹등키 = 요청마다 유니크한 값

요청 (먹등키 abc-123)

재시도 (먹등키 abc-123)

1번만 처리
같은 응답

같은 요청에는 같은 응답을 보장
→ 요청 쪽이 **안심하고 재시도**할 수 있다

→ 요청 쪽은 “모름”을 관리하고, 제공 쪽은 먹등키로 재시도를 안전하게

4-way handshake — 각자 따로 “다 보냈어”

②와 ③이 따로인 이유

FIN을 받아도 서버는
아직 보낼 데이터가 남았을 수 있다

책의 “우아한 끊기”

내 출력부터 닫고,
상대가 닫을 때까지 기다린다

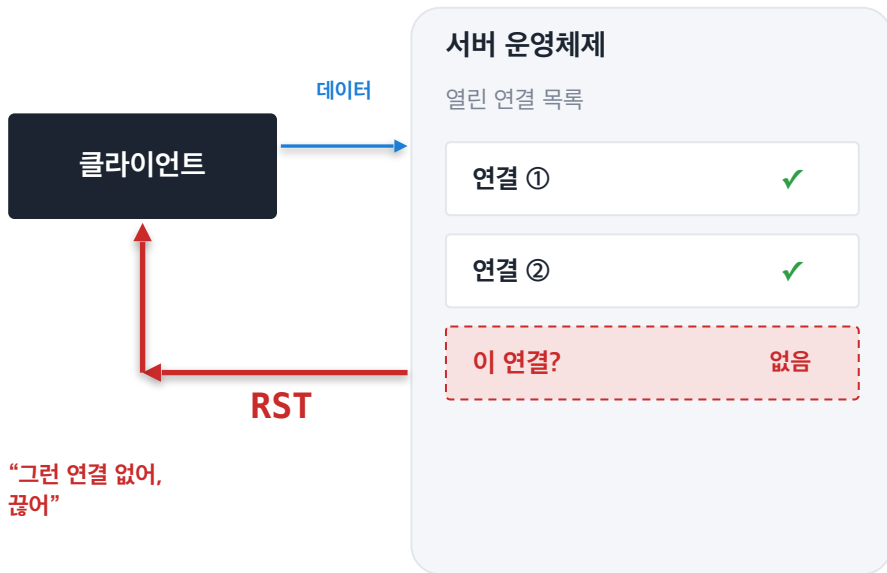
TIME_WAIT

먼저 끊은 쪽은 잠깐 기다렸다 닫힘
(늦게 도착한 패킷 처리용)



→ TCP는 양방향 → 각자 따로 “나 이제 보낼 거 없어”를 말해서 4단계

RST — 운영체제의 “이 연결 모르는데?”



FIN 우아한 종료

- 애플리케이션이 정리하며 보냄
- “나 보낼 거 다 보냈어”

RST 갑작스러운 종료

- 운영체제가 자동으로 보냄
- 앱이 일부러 보내는 경우는 드물

→ 목록에 없는 연결로 데이터가 오면 **운영체제가 RST**를 돌려보낸다

RST가 오는 세 가지 상황

상황 1 · 가장 흔함

keep-alive 타임아웃

한동안 요청이 없어 서버가 연결을 닫음
→ 서버 OS 목록에서 사라짐
클라이언트는 모르고 폴의 그 연결로 새 요청
서버 OS: “이미 닫았는데?”

Connection reset by peer

상황 2

서버 프로세스가 갑자기 죽음

서버 앱이 크래시 · 강제 종료
OS가 그 앱의 연결들을 정리
클라이언트가 그 연결로 데이터 전송
서버 OS: “이 연결 주인이 없어”

RST

상황 3

닫힌 포트에 연결 시도

서버 8080 포트에 아무 앱도 없음
클라이언트가 8080으로 연결 요청
서버 OS: “여기 듣는 앱 없어”

Connection refused

→ 상황 1은 앞에서 본 “하필 그 순간” 문제 — 클라이언트 idle 타임아웃을 더 짧게

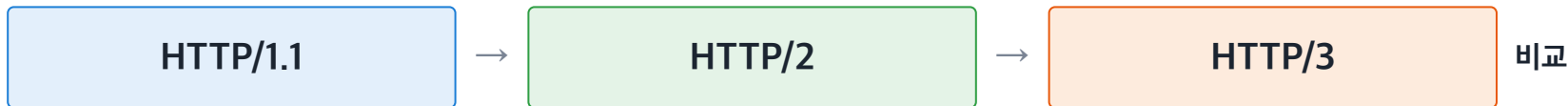
PART 4

HTTP/2 · HTTP/3

10장 · p.287 ~ 294 + 부록 HTTP/3

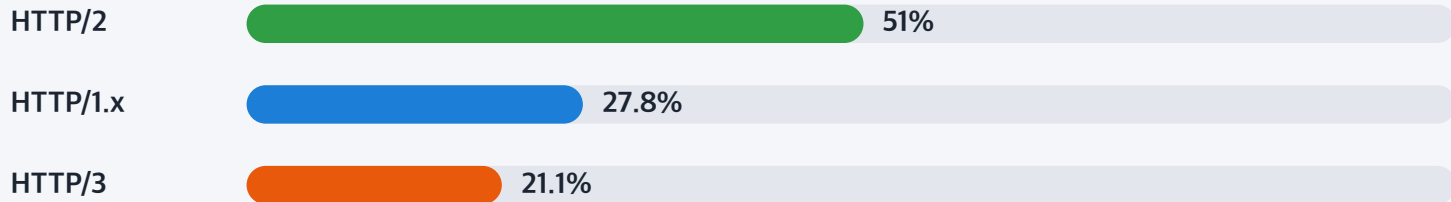
HTTP/1.1 · HTTP/2 · HTTP/3 — 무엇이 달라졌나

학습 목표



사용 통계

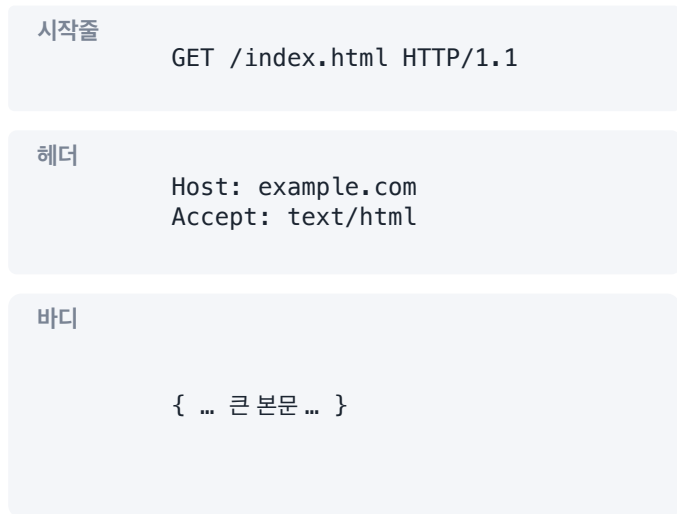
Cloudflare Radar · 2026년 1분기



→ 이미 HTTP/2가 절반 이상, HTTP/3도 다섯 중 하나

내용은 그대로, 프레임에 나눠 담는다

HTTP/1.1 메시지

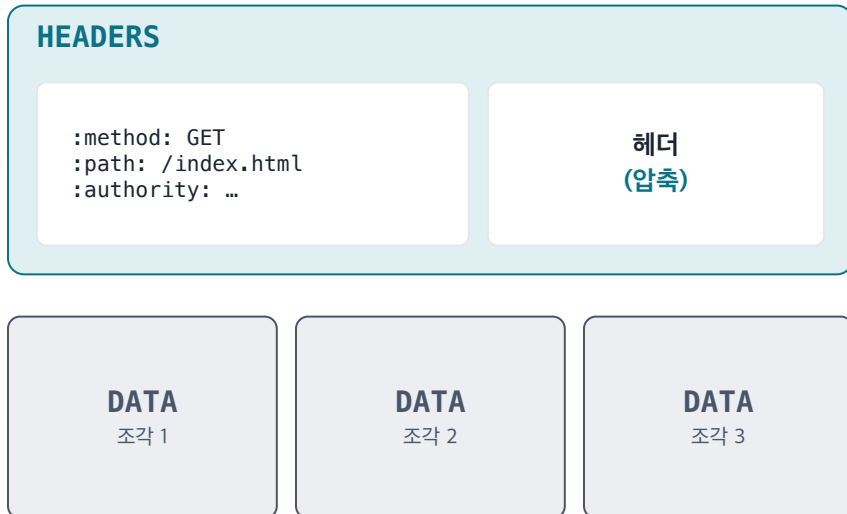


콜론 헤더로

압축

여러 개로

HTTP/2 프레임



→ 요청 하나 = **HEADERS 1개** + **DATA 여러 개**

프레임 구조 — 9바이트 헤더 + 페이로드

프레임 헤더 · 9바이트 (고정)



Length	Type	Flags	Stream ID
페이로드 길이	프레임 종류 DATA · HEADERS ...	부가 정보 예) END_STREAM = 이 스트림 끝	어느 스트림의 프레임인지 → 멀티플렉싱의 핵심

→ 모든 프레임에 **스트림 ID**가 붙는다 — 섞여 보내도 다시 모을 수 있는 이유

프레임 종류

자주 쓰는 것

DATA

바디를 담는다

HEADERS

헤더를 담는다

CONTINUATION

헤더가 길 때 이어서

연결 관리용

SETTINGS

설정을 주고받음

PING

연결이 살아 있는지 확인

GOAWAY

연결 종료

WINDOW_UPDATE

흐름 제어 (속도 조절)

RST_STREAM

요청 하나만 취소

사실상 안 쓰는 것

PRIORITY

폐기됨

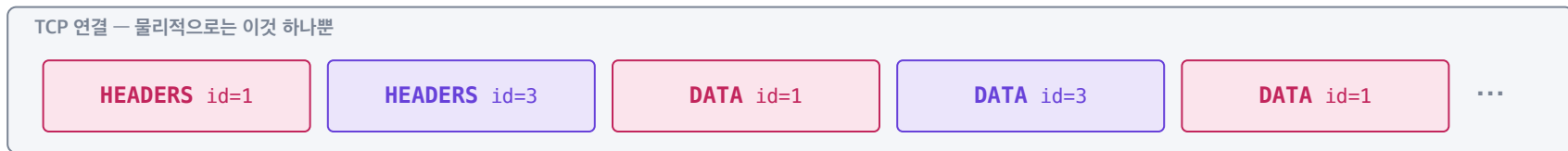
PUSH_PROMISE

서버 푸시용 — 브라우저에서 제거

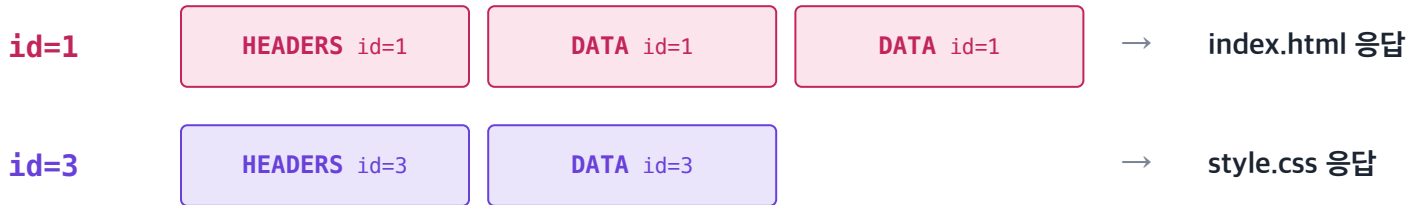
→ **HEADERS · DATA** 가 주인공, 나머지는 연결을 관리한다

스트림 — ID로 구분되는 가상의 통로

스트림 = 요청 하나와 그 응답이 오가는 **일회용 통로** (열고 → 요청 → 같은 스트림으로 응답 → 닫힘)



▼ 받는 쪽은 ID별로 모아서 본다



→ 스트림은 따로 있는 선이 아니라, **같은 ID를 가진 프레임들의 묶음**

TCP는 보낸 순서대로 받게 해 준다

1 보낼 때 번호를 붙인다



데이터를 잘게 나눠
각 조각에 “몇 번째 바이트부터”인지
번호를 붙인다

2 받는 쪽은 순서대로 재조립



버퍼에 보관만

앱에는 1번까지만 넘긴다
2번이 오면 2 · 3 · 4 한꺼번에

3 빠진 건 다시 보내게 한다

ACK “2번부터 못 받았어”

↓ 확인이 안 오면



재전송

→ a 요청 → b 요청을 보내면, 서버는 반드시 a → b 순서로 받는다

HTTP/1.1의 문제 — 응답은 순서대로만

한 연결로 연달아 (파이프라이닝)



응답 어디에도 “/a에 대한 답” 표시가 없다

→ 먼저 온 응답 = 먼저 보낸 요청의 답 (순서로만 짝 맞춤)

/b를 먼저 보내 버리면?

클라이언트는 /b의 내용을

/a의 결과로 착각한다

서버 쪽 시간 흐름

/a 처리

/b 처리

응답

준비됐는데 대기...

/a

/b

앞(Head)이 막히면 뒤가 다 기다린다

HOL (Head-of-Line) Blocking

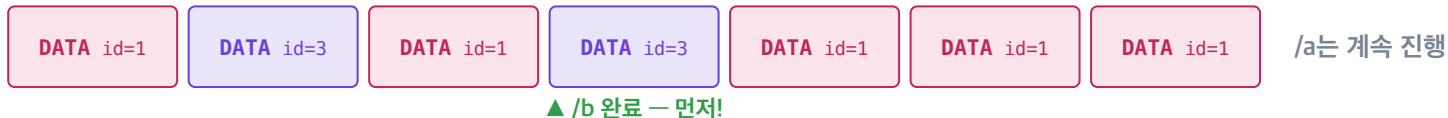
→ 응답에 짝 표시가 없어서 **순서를 지켜야 하고**, 그래서 **앞이 막히면 다 막힌다**

멀티플렉싱 — 한 연결 위에서 여러 스트림을 동시에

HTTP/1.1 — 순서대로



HTTP/2 — 프레임 단위로 쪼개서



1

요청 · 응답을 프레임 단위로 잘게 쪼개 보내고
받는 쪽이 ID별로 다시 조립

2

큰 응답이 진행 중이어도
작은 응답이 사이에 끼어 먼저 끝낼 수 있다

→ 응답을 다른 스트림 · 프레임 단위로 보내서 HTTP 레벨의 HOL Blocking 해결

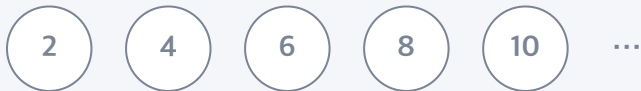
스트림 ID 규칙

클라이언트가 연 스트림
홀수



지금은 사실상
홀수 스트림만 쓰인다

서버가 연 스트림
짝수



서버 푸시용이었는데
브라우저에서 제거됨 → 사실상 안 씀



항상 이전보다 큰 ID



한 번 쓴 ID는 재사용 불가



ID가 다 떨어지면 연결을 새로

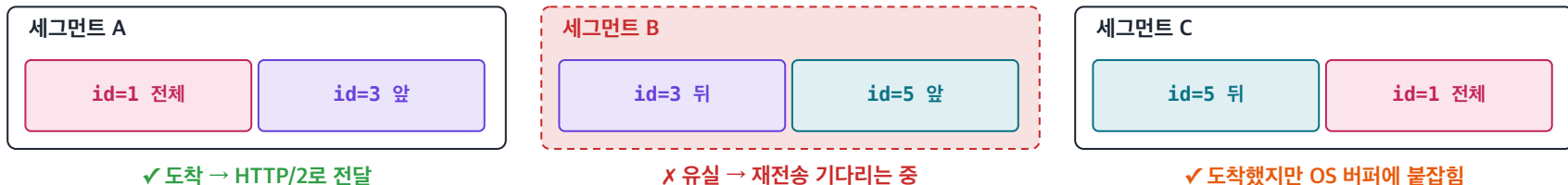
→ ID는 **한 방향으로만 증가** — 다 쓰면 새 연결

왜 HTTP/3? — TCP 계층의 HOL Blocking

HTTP/2가 보는 것 — 스트림 ID가 붙은 프레임들



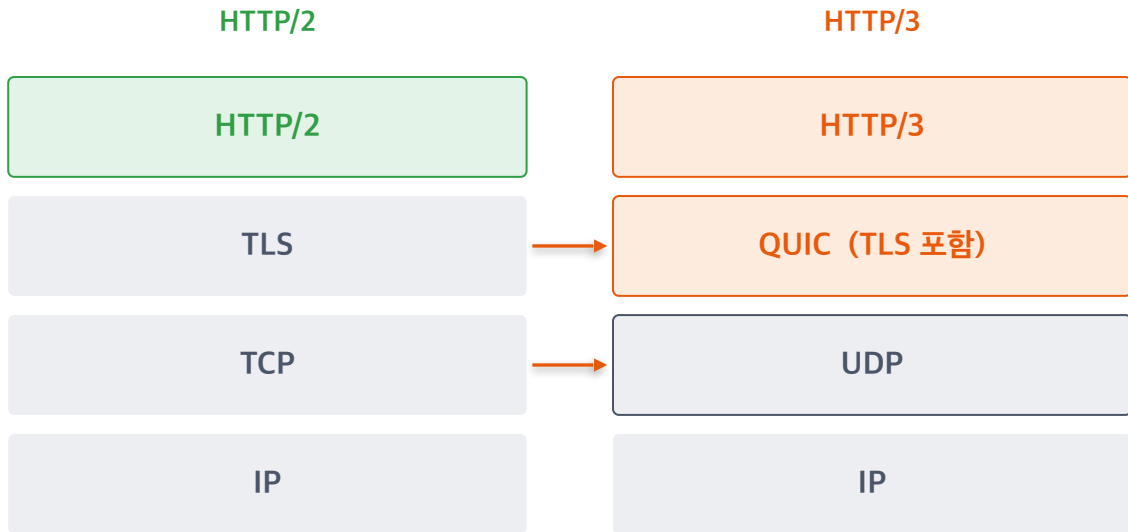
TCP가 보는 것 — 바이트 한 줄을 자른 세그먼트



C 안의 id=1 프레임은 B와 상관없는데도 → TCP는 바이트 순서를 지켜야 해서 HTTP/2는 못 받고 기다린다

→ HTTP/2가 스트림을 나눠도 TCP는 바이트 한 줄로만 본다 = TCP 계층 HOL Blocking

HTTP/3 — TCP 대신 QUIC 위에서 동작하는 HTTP



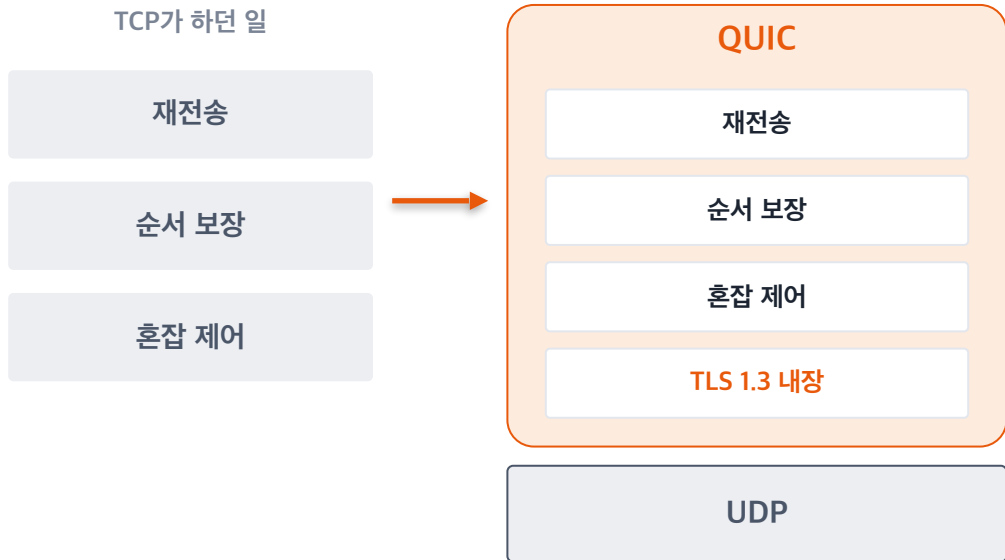
HTTP/2의 **남은 문제**를 풀려고
아래를 바꿨다

HTTP의 **의미**는 그대로
메서드 · 헤더 · 스트림 개념 유지

바뀐 건 **데이터를 나르는 방식**

→ 위(HTTP)는 그대로, 아래가 **TLS + TCP** → **QUIC + UDP** 로 바뀌었다

QUIC — UDP 위에 TCP의 일을 새로 만든 것



왜 TCP를 안 고치고?

TCP는 운영체제와 네트워크
깊이 박혀 있어서 고치기 어렵다

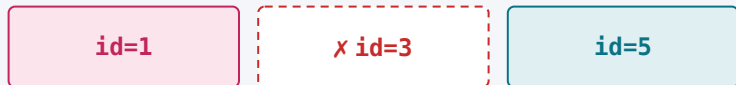
그래서 아무것도 안 하는
UDP 위에
필요한 기능을 직접 올렸다

→ **QUIC** = **UDP** + (재전송 · 순서 보장 · 혼잡 제어 · TLS)

좋아진 점 ① TCP 계층 HOL Blocking 해결

HTTP/2 — TCP

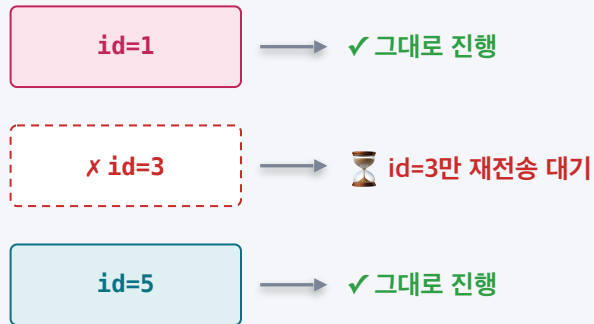
순서 보장을 연결 전체로



id=3 패킷 하나가 유실되면
상관없는 id=1 · id=5까지 멈춘다

HTTP/3 — QUIC

순서 보장을 스트림별로

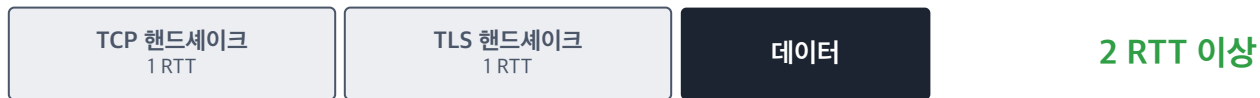


→ QUIC은 스트림을 알고 있어서 잃어버린 스트림만 기다린다

좋아진 점 ② 연결이 빠르다

TCP + TLS

따로따로 핸드셰이크



QUIC

TLS 1.3 내장



QUIC 재접속

전에 접속했던 서버



연결 시작

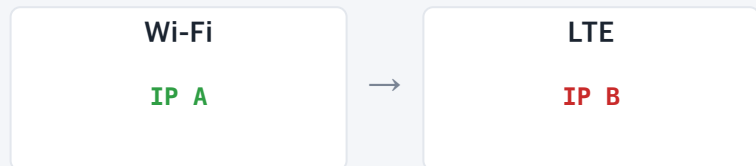
시간 →

→ TLS를 품어서 1번 왕복, 다시 오면 바로 데이터 (0-RTT)

좋아진 점 ③ 네트워크가 바뀌어도 연결 유지

TCP

IP + 포트로 연결을 구분



✗ 연결 끊김 → 다시 연결

기준(IP)이 바뀌었으니 다른 연결

QUIC

연결 ID로 연결을 구분



✓ IP가 바뀌어도 이어감

기준(연결 ID)은 그대로니 같은 연결

→ 모바일 (Wi-Fi ↔ LTE 전환) 환경에서 특히 유리

2026년 현재 — 어디까지 왔나

지원 범위

주요 브라우저 모두 지원

이미 쓰는 대형 서비스

Google

Cloudflare

Meta

연결 방식

1 처음엔 HTTP/2로 접속

2 응답 헤더로 “HTTP/3도 돼”

`Alt-Svc: h3=":443"`

3 다음부터 HTTP/3로

단점

UDP를 막는 네트워크

일부 회사 · 기관 → HTTP/2로 되돌아감

서버 CPU 사용량

TCP보다 높은 편

→ 이미 널리 쓰이지만, 항상 HTTP/2라는 안전망과 함께

한눈에 비교

	HTTP/1.1	HTTP/2	HTTP/3
전송 계층	TCP	TCP	QUIC (UDP)
데이터 단위	메시지 통째로	프레임	프레임
동시 요청	순서대로	멀티플렉싱	멀티플렉싱
HOL Blocking	HTTP 레벨 있음	TCP 레벨 없음	해결 (스트림별)
헤더	텍스트 그대로	압축	압축
연결 수립	TCP + TLS	TCP + TLS	1 RTT · 0-RTT
네트워크 변경	끊김	끊김	유지 (연결 ID)

→ HTTP/2는 HTTP 레벨을, HTTP/3는 그 아래 전송 계층까지 고쳤다